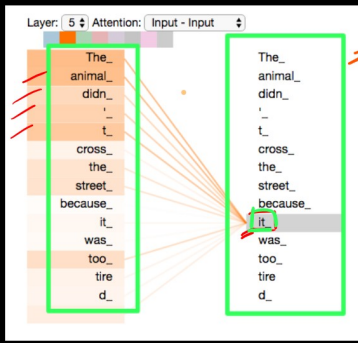




Self Attention

$$[it] = w_1 * the + w_2 * animal + \dots + w_n d$$

↑ ↑ ↑

Self Attention

S1 → money bank grow
 Target word: bank
 $bank = w_1 \cdot money + w_2 \cdot bank + w_3 \cdot grow$

money: $w_1 \cdot money + w_2 \cdot bank + w_3 \cdot grow$
bank: $w_1 \cdot money + w_2 \cdot bank + w_3 \cdot grow$
grow: $w_1 \cdot money + w_2 \cdot bank + w_3 \cdot grow$

Self Attention

weight

S2 → river bank flow
 Target word: bank
 $bank = w_1 \cdot river + w_2 \cdot bank + w_3 \cdot flow$

river: $w_1 \cdot river + w_2 \cdot bank + w_3 \cdot flow$
bank: $w_1 \cdot river + w_2 \cdot bank + w_3 \cdot flow$
flow: $w_1 \cdot river + w_2 \cdot bank + w_3 \cdot flow$

weight

Adjust

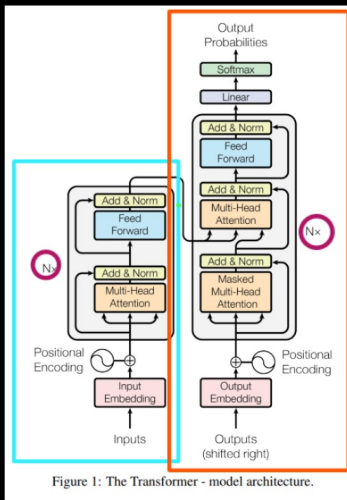
Context

These weights come from:

$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$

= Z

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



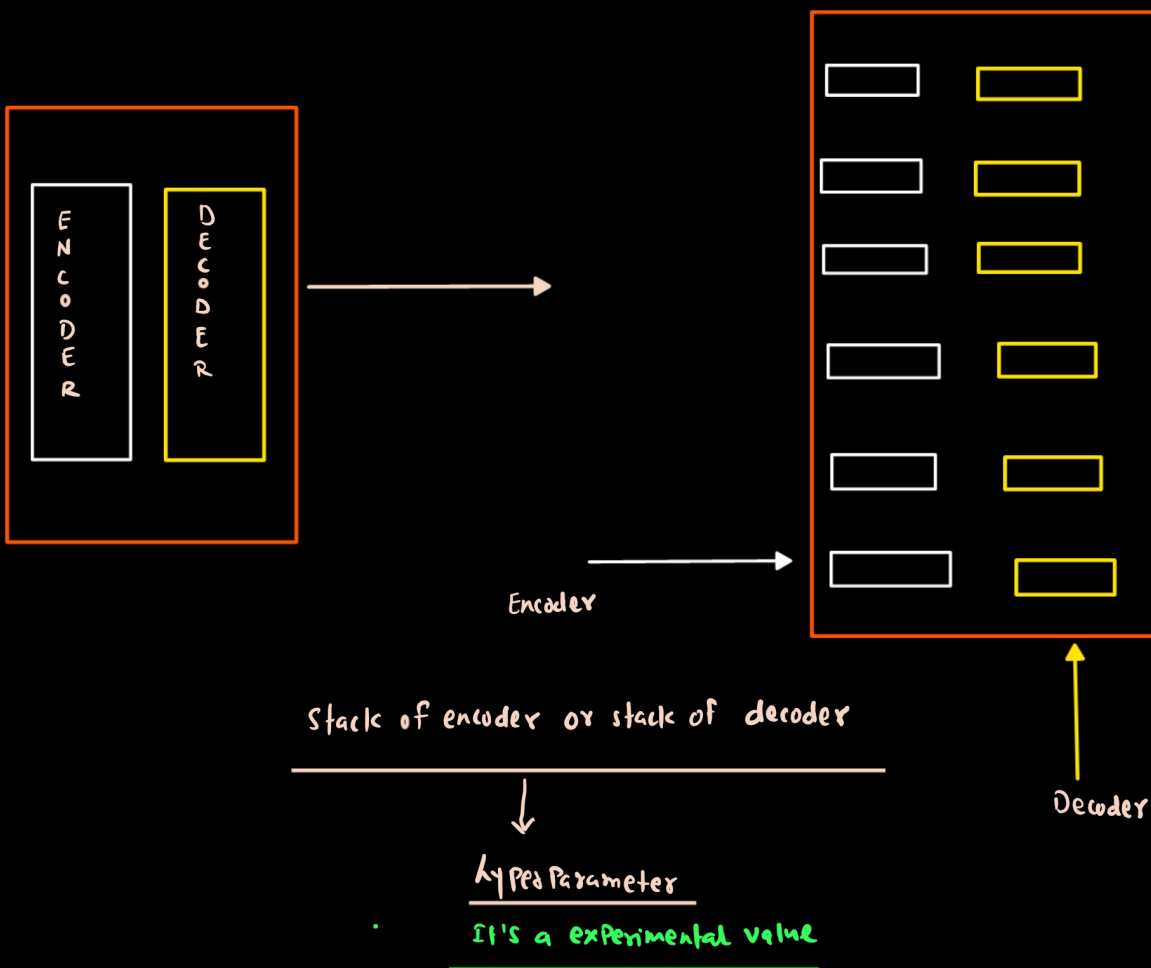
- Encoder Side:
1. input sequence
 2. input embedding
 3. positional encoding
 4. self-attention, multi-head attention
 5. add and layer norm
 6. feed forward network (artificial neural network)
 7. then again addition (residual connection) and layer normalization
 8. Nx (multiple encoder block (in original research paper there was 6 block and all the block were identical means in every block same process happening))

→ stack of encoder & decoder
 = why to keep multiple encoder & decoder?

Text → Tokenization → Embedding Layer → Positional Encoding → Self-Attention → FFN → Output

the embedding layer is essentially similar to a Word2Vec model; the only difference is that it is not a pretrained model.

In the Transformer architecture, the embedding layer does not use pretrained Word2Vec by default. Instead, it uses a trainable neural network layer that works in a similar way to Word2Vec, meaning it learns vector representations of words during training while the data is being passed into the Transformer encoder.



Why 6 Encoder Blocks?
Important: 6 is NOT fixed

← Hyperparameter value (Experimental)

From paper: Original Transformer used
6 encoder layers ✓
6 decoder layers ✓

multiple encoder & multiple decoder

Reason

← why the stack is imp

Stacking layers helps:

capture deeper relationships
understand long context
build hierarchical meaning

Data → very long

↳ ~~vertically~~

Modern models:

BERT Base → 12 layers

BERT Large → 24 layers

GPT → 12-96+ layers

transformer

llama / mistral → {stack is going deeper}

decoder part of transformer

Each layer:

refines representation more

Layer 1 → basic word relation

Layer 3 → phrase meaning

Layer 6 → sentence understanding

Layer 12 → deep context

ENCODER

~~E2E~~

2 main → Process

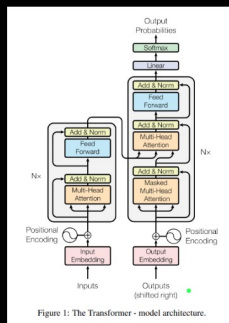
✓ Feed-forward NN

NN

✓ Self-Attention

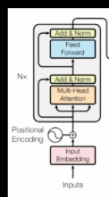
(Q, K, V)

Position encoding

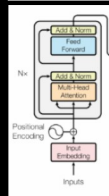


Why the stack of the encoder or decoder ?

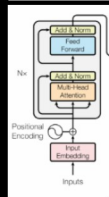
output



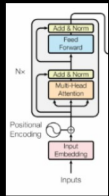
6



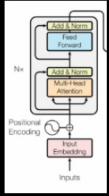
5



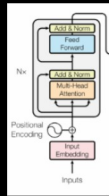
4



3



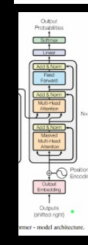
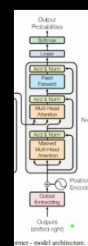
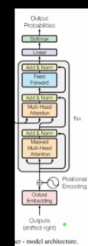
2



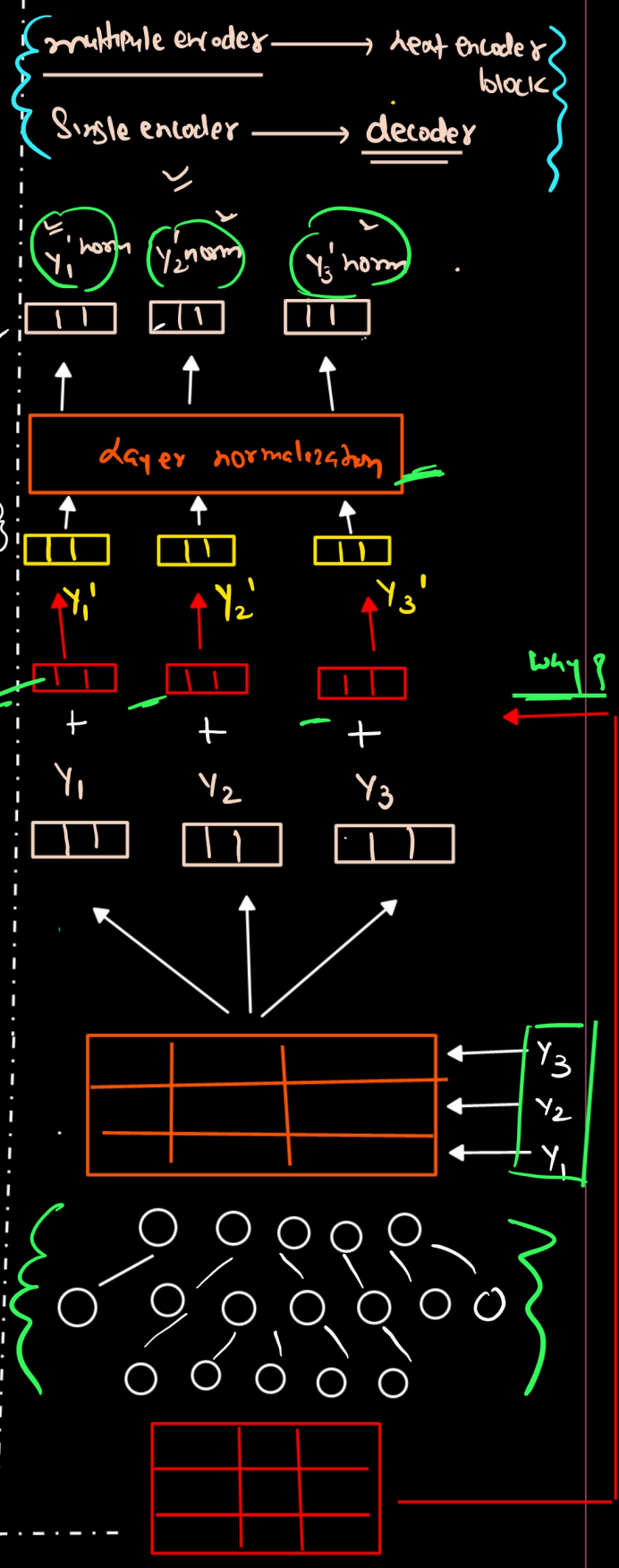
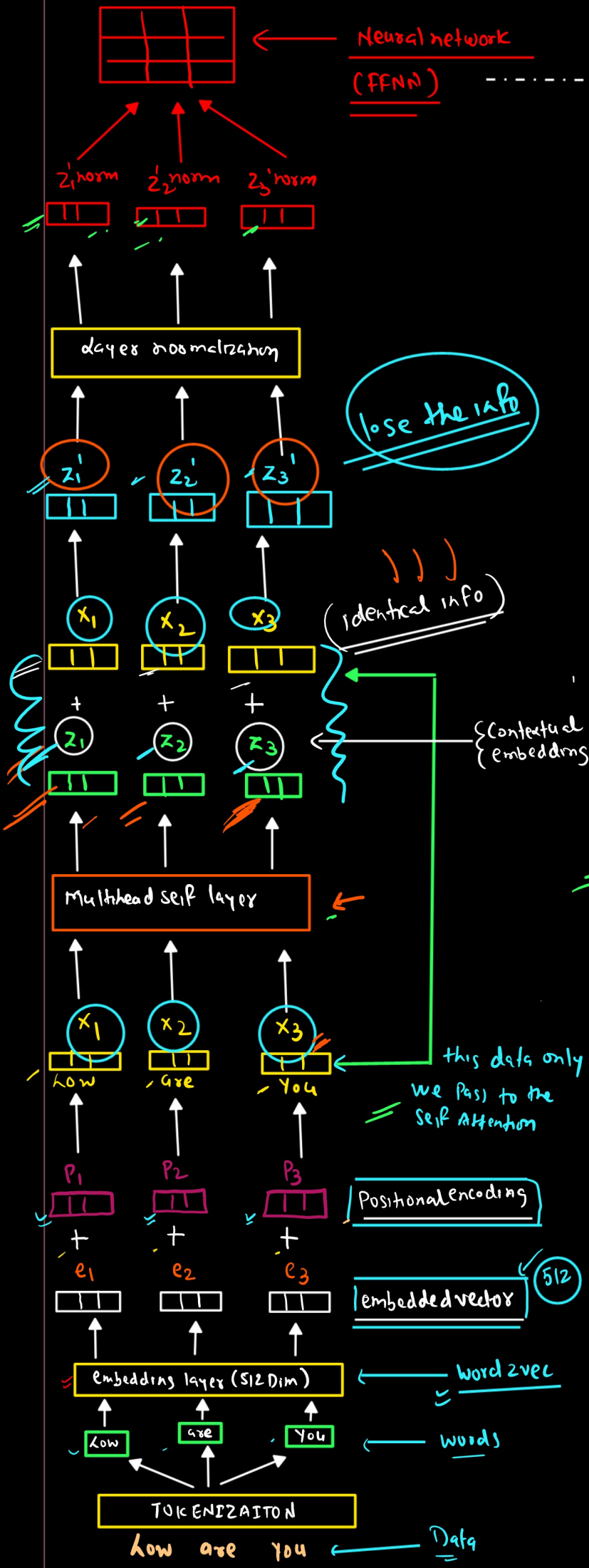
1

Identical $\rightarrow$ SAME

96



Input

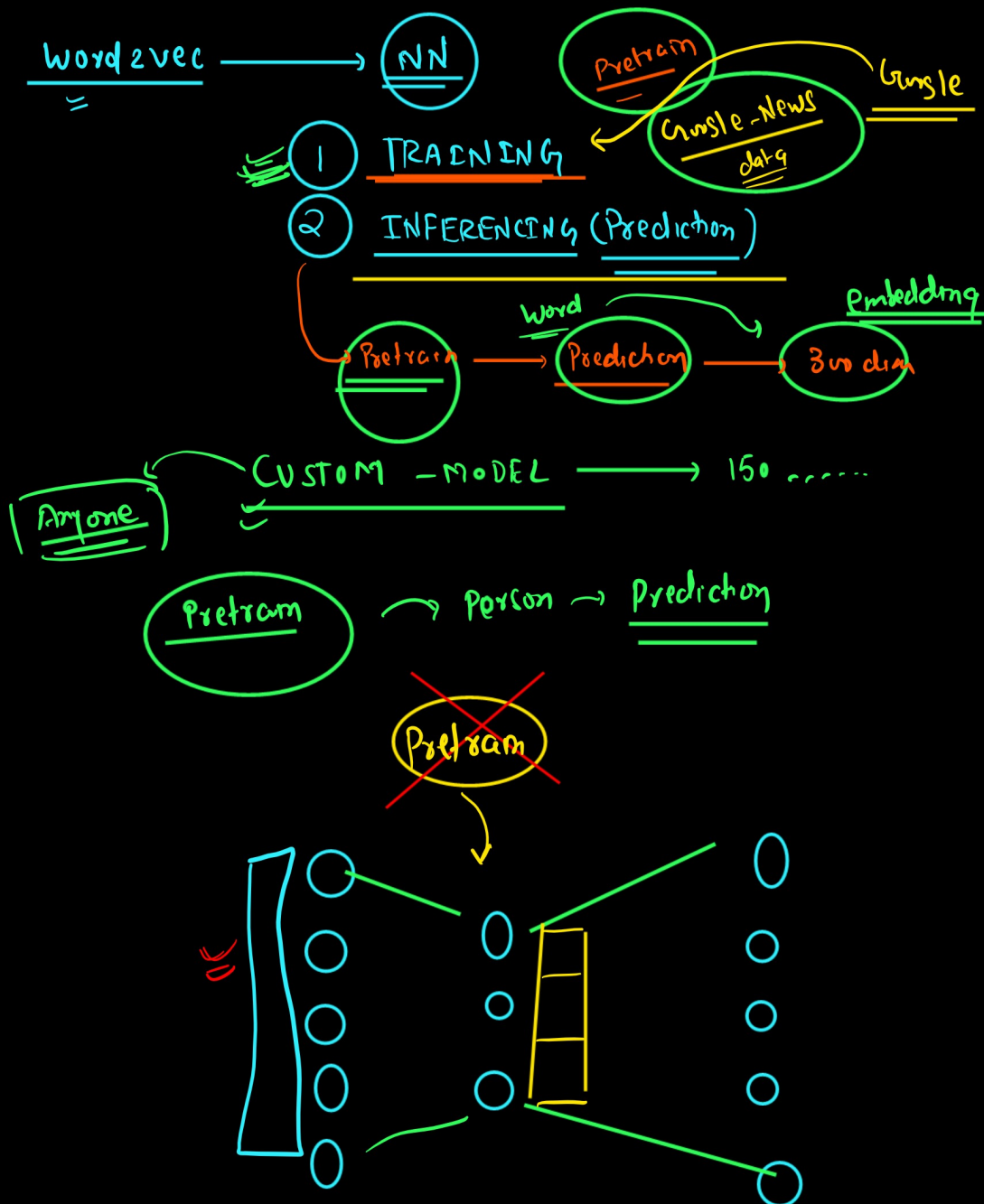


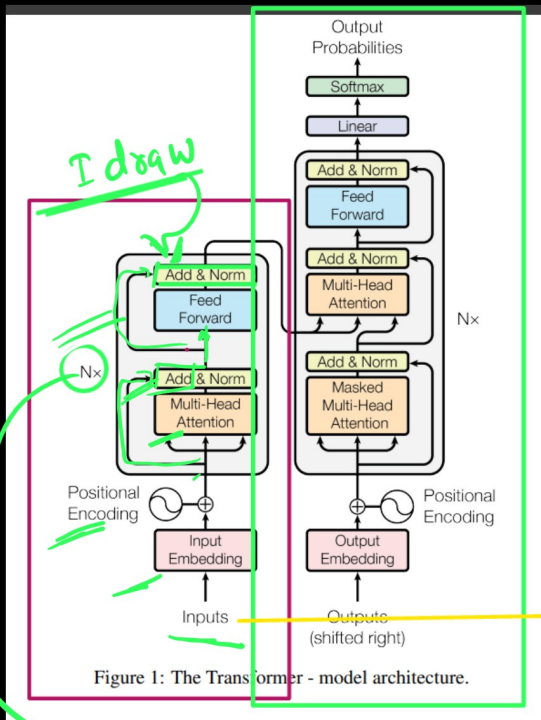
word2vec

Text → Tokenization → Embedding Layer → Positional Encoding → Self-Attention → FFN → Output

the embedding layer is essentially similar to a Word2Vec model; the only difference is that it is not a pretrained model.

In the Transformer architecture, the embedding layer does not use pretrained Word2Vec by default. Instead, it uses a trainable neural network layer that works in a similar way to Word2Vec, meaning it learns vector representations of words during training while the data is being passed into the Transformer encoder.





Did you understand data?

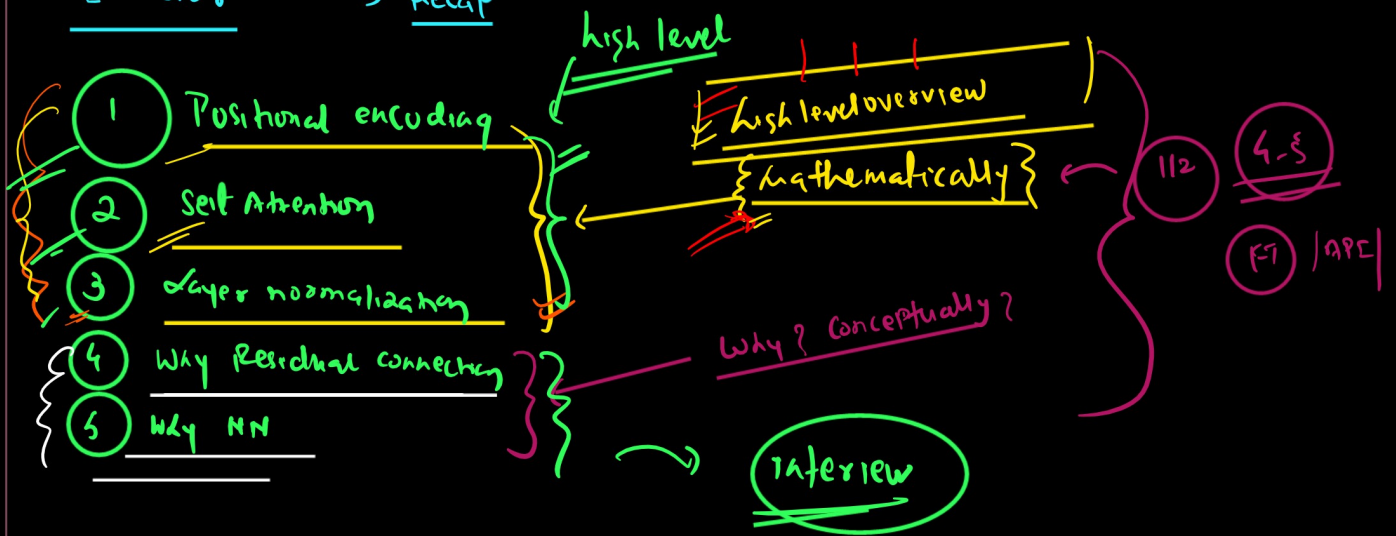
multiple stack of encoder

encoder

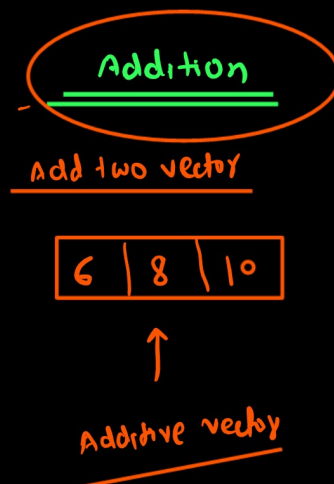
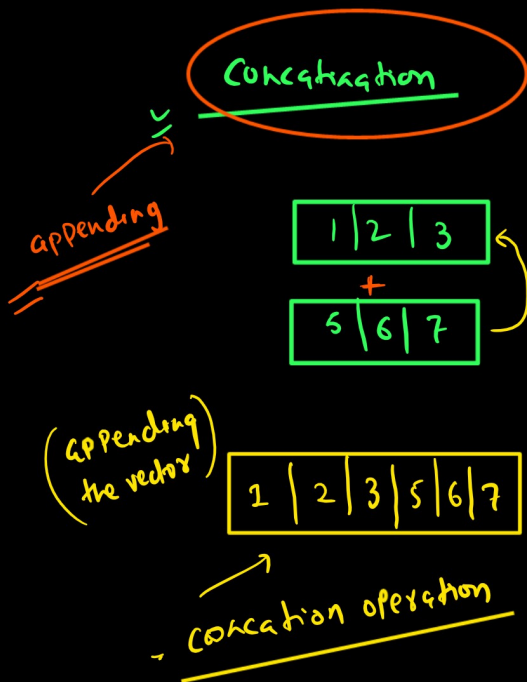
clarify

- 1 Positional encoding Q_1
- 2 Attention (self attention) Q_2
- 3 layer normalization Q_3
- 4 what is Residual connection Q_4
- 5 why they used it? Q_5
- 6 why they used NN Q_6

Encoder → Recap



Decoder → Cross Attention



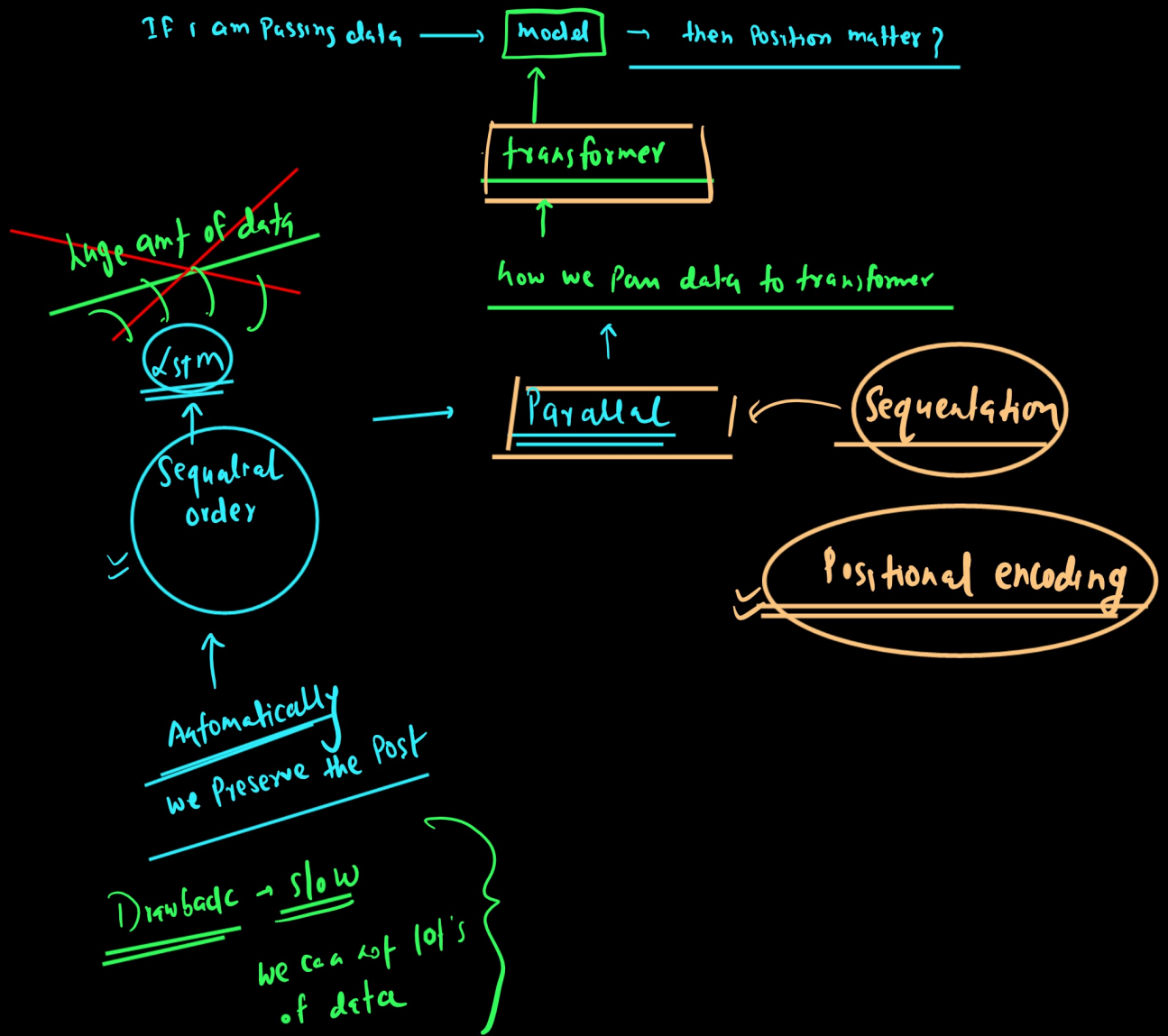
Sunny is taking GenAI class & teaching transformer

wordseq is rmp

Sunny is GenAI taking teaching & transformer

meaning is spring to be change

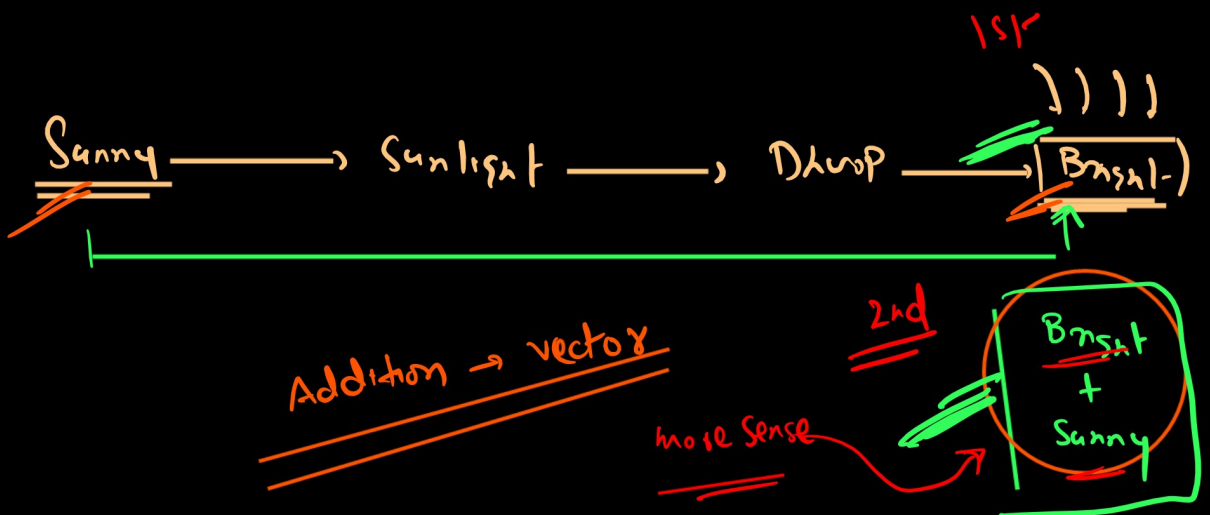
problem



Residual connection $\rightarrow$ skip connection

Resnet (CNN)

keep the information As it is
keep the identical info?



Why Residual Connection is used?

Deep networks suffer from:

Vanishing gradients ↘

Training becomes unstable

Information gets lost

Solution: Residual Connection

Instead of learning:

"completely new transformation"

Model learns:

"small improvement over input"

Why it helps

Better gradient flow ↗

Faster training

Prevents information loss

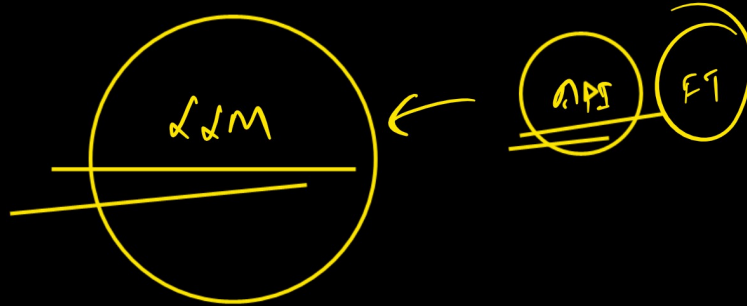
Enables deep stacking (like 6, 12, 24 layers)

weights are not going to be

Retaining a context.

weights

deep



Why Feed Forward Neural Network (FFN)?

Self-attention only:

learn information across tokens

FFN role

Each token independently passes through:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

hidden o/p
512 → 2048 → 512

original
Research
Paper

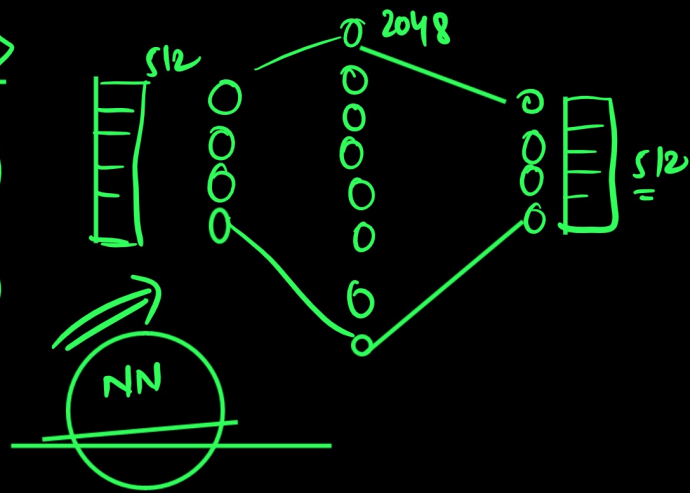
Key role

Adds non-linearity

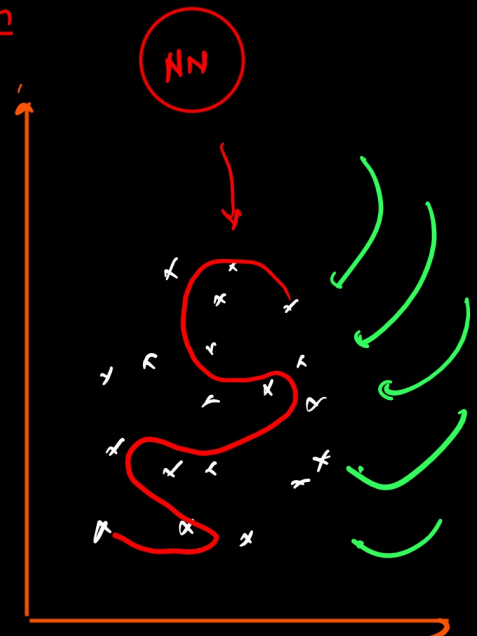
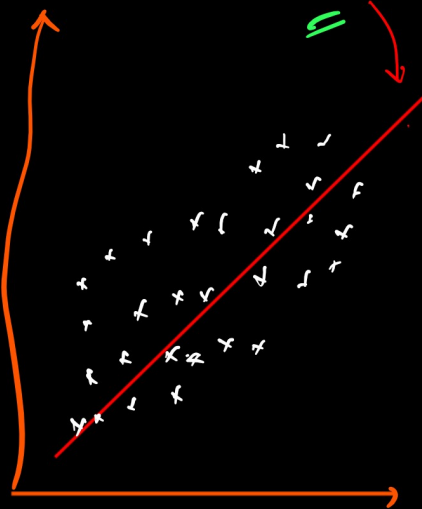
Learns complex patterns

decode

word level Relation?



Linear model ← Linear Regression



{ Non-Linear Pattern from the
data → Complex Pattern }

Self-Attention → word level attention → Complex Pattern

decode info more



Why 6 Encoder Blocks?
Important: 6 is NOT fixed

From paper: Original Transformer used
6 encoder layers
6 decoder layers

Reason
Stacking layers helps:
capture deeper relationships
understand long context
build hierarchical meaning

Modern models:
BERT Base → 12 layers
BERT Large → 24 layers
GPT → 12–96+ layers

Each layer:
refines representation more
Layer 1 → basic word relation
Layer 3 → phrase meaning
Layer 6 → sentence understanding
Layer 12 → deep context